Anomaly Detection in Time Series with Robust Variational Quasi-Recurrent Autoencoders

Tung Kieu[1], Bin Yang[1*], Chenjuan Guo[1], Razvan-Gabriel Cirstea[1], Yan Zhao[1], Yale Song[2], Christian S. Jensen[1]
[1]Aalborg University, Denmark [2]Microsoft Research, USA
[1]{tungkvt, byang, razvan, yanz, cguo, csj}@cs.aau.dk, [2]yalesong@microsoft.com

***Abstract*—We propose variational quasi-recurrent autoencoders (VQRAEs) to enable robust and efficient anomaly detection in time series in unsupervised settings. The proposed VQRAEs employs a judiciously designed objective function based on robust divergences, including α, β, and γ-divergence, making it possible to separate anomalies from normal data without the reliance on anomaly labels, thus achieving robustness and fully unsupervised training. To better capture temporal dependencies in time series data, VQRAEs are built upon quasi-recurrent neural networks, which employ convolution and gating mechanisms to avoid the inefficient recursive computations used by classic recurrent neural networks. Further, VQRAEs can be extended to bi-directional BiVQRAEs that utilize bi-directional information to further improve the accuracy. The above design choices make VQRAEs not only *robust* and thus *accurate*, but also *efficient* at detecting anomalies in streaming settings. Experiments on five real-world time series offer insight into the design properties of VQRAEs and demonstrate that VQRAEs are capable of outperforming state-of-the-art methods.**

I. Introduction

We study anomaly detection for time series, where anomalies are observations that are significantly different from the majority of observations in a sequence of observations that are ordered by time. This functionality is essential in many application domains [30], [35], as it enables us to discover exceptions of the underlying systems, thus enabling, e.g., warnings in advance that prevent potential malfunctions and cleanings [50], [52]. In this paper, we study *unsupervised anomaly detection*, where labels indicating anomalies are not available for training. Indeed, such labels are rarely available in real world applications [1], [7].

Recently, deep learning methods achieve very competitive results in unsupervised anomaly detection. Such methods often rely on a so-called autoencoder (AE) architecture that consists of an encoder and a decoder [24]. In the encoding phase, the encoder transforms input data into a compact, hidden representation. In the decoding phase, the decoder reconstructs the input from the hidden representation. Finally, the differences between the input and the reconstructed input are used as indicators for anomalies. The intuition behind is that the hidden representation is compact, thus being able to capture only regular patterns in the input data. As a result, the reconstructed input is unlikely to capture anomalies that are often beyond regular patterns. Thus, large differences between the input and the reconstructed input are appropriate indicators for anomalies. In addition, this process does not rely on anomaly labels, thus being unsupervised.

There exist a few improvements to classic AEs. First, variational autoencoders (VAEs) improve classic AEs by introducing *stochastic* hidden representations that capture distributions of normal patterns in the input data, which offer possibilities to model unseen normal patterns from the input data. In contrast, classic AEs employ *deterministic* hidden representations that merely model normal patterns that appear in the input data.

Second, recurrent autoencoders (RAEs) improve classic AEs by using recurrent neural networks, which are known to be able to capture temporal dependencies in sequence data, such as natural languages and time series, to build the encoders and decoders. In contrast, classic AEs use feed-forward neural networks, which fail to capture temporal dependencies. Thus, RAEs fit better for time series anomaly detection. Combining VAEs and RAEs gives rise to Variational Recurrent Autoencoders (VRAEs) [11], [42], which achieves the state-of-the-art accuracy for time series anomaly detection.

However, VRAEs still suffer from three limitations. First, they do not support fully unsupervised anomaly detection. Although they do not require anomaly labels indicating the anomalies in the training data, they do assume that the training data includes only clean data without anomalies. However, in real-world unlabeled data sets, it is often impossible to make sure that only clean data is included. When training with contaminated data, i.e., data that may already include anomalies, the obtained latent representations capture the features of both clean and anomalous observations, thus making the reconstructed observations include anomalies after decoding. In other words, anomalies in training data may pollute and negatively affect the latent representation such that some anomalies may have small reconstruction errors and thus are difficult to separate from clean data. This calls for solutions that are able to prevent anomalies in the training data from contaminating the latent representation.

Second, the computation time for RAEs is high due to the recursive computations in recurrent architectures, e.g., RNNs. This limitation prevents VRAEs from time-critical applications and systems where the computation time is an important consideration.

Third, VRAE limits to uni-directional computations due to a "lock" structure. As a result, VRAE is able to consider temporal

*Corresponding author.

dependencies from only one direction. However, many studies in sequence modeling and learning suggest that bi-directional computations contribute to improved accuracy [4], [41]. Thus, this calls for a novel architecture without the "lock" structure such that bi-directional computations are possible.

To contend with the above limitations, we propose Variational Quasi-Recurrent Neural Network Autoencoders (VQRAEs) for anomaly detection in time series data. First, VQRAEs achieve robustness by replacing the standard log-likelihood objective function, which is often used in existing VAEs [42], by robust objective functions, including α, β, and γ-divergence objective functions. The α, β, and γ-divergence functions model heavy-tailed distributions, such that abnormal observations can be better distinguished from normal observations. This avoids abnormal observations to contaminate the latent representation in the autoencoder, thus making the latent representation being able to model only the normal observations, which significantly improves anomaly detection robustness and thus also accuracy.

Second, VQRAEs are built upon Quasi-Recurrent Neural Networks (QRNNs), achieving high training efficiency. Specifically, VQRAEs employ non-recursive gating mechanisms to capture temporal dependencies, and thus are more efficient than recurrent architectures such as LSTMs and GRUs.

Third, VQRAEs decouple stochastic hidden state with deterministic hidden states such that bi-directional computations are possible. Thus, we propose a bi-directional VQRAE, denoted as BiVQRAE, that is able to capture temporal dependencies from both directions, contributing to significantly improved accuracy.

In summary, VQRAEs are more powerful than existing AEs, VAEs, RAEs, and VRAEs because VQRAEs are robust to anomalies in training data, are able to explicitly capture temporal dependencies with bi-directional mechanism, are able to model unseen patterns by using stochastic latent representations, and are efficient. No other model can achieve all properties. Table I summarizes autoencoder-based anomaly detection methods, suggesting that VQRAEs address limitations of existing solutions.

TABLE I: Autoencoder-based Anomaly Detection Comparison

	Stochastic	*Temporal Dependency*	*Robustness*	*Efficiency*	*Bi-directional Modeling*
AE	✗	✗	✗	✓	✗
VAE	✓	✗	✗	✓	✗
RAE	✗	✓	✗	✗	✓
VRAE	✓	✓	✗	✗	✗
VQRAE	✓	✓	✓	✓	✓

To the best of our knowledge, this is the first study that proposes efficient and robust variational autoencoders for time series anomaly detection. More specifically, we make three contributions.

- We propose a robust objective function for training on contaminated time series, which is suitable for fully unsupervised settings.
- We propose an efficient variational quasi-recurrent autoencoder model and its bi-directional version for time series anomaly detection.
- We report on an extensive empirical study using both univariate and multivariate time series that offers insight into the design properties of the proposed method.

The rest of the paper is organized as follows. Section 2 covers preliminaries and formulates the problem. Section 3 proposes the two models. Experimental results are reported in Section 4, and related work is covered in Section 5. Finally, Section 6 concludes the paper.

II. Preliminaries and Problem Definition

A. Time Series

A time series $\mathcal{T} = \langle \mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_C \rangle$ is a time-ordered sequence of C observations, where C is the length of the time series (i.e., the number of observations). Each observations $\mathbf{s}_i = (s_i^{(1)}, s_i^{(2)}, \ldots, s_i^{(k)})$ is a k-dimensional vector representing k features of an entity at time point t_i, where $1 \leq i \leq C$. When $k = 1$, the time series is *univariate*; and when $k > 1$, the time series is *multivariate*.

B. Anomaly Detection in Time Series

Given a time series $\mathcal{T} = \langle \mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_C \rangle$, we aim at computing an anomaly score $\mathcal{AS}(\mathbf{s}_i)$ for each vector $\mathbf{s}_i$ such that the higher $\mathcal{AS}(\mathbf{s}_i)$ is, the more likely it is that vector $\mathbf{s}_i$ is an anomaly.

C. Autoencoders (AEs)

An Autoencoder (AE) consists of an encoder and a decoder, where each is a neural network [20], [24]. More specifically, the encoder transforms an input $\mathbf{s} \in \mathbb{R}^k$ to an intermediate and compact deterministic vector $\mathbf{h} \in \mathbb{R}^m$, where $m < k$. Then, the decoder transforms the intermediate vector $\mathbf{h}$ to an output vector $\hat{\mathbf{s}} \in \mathbb{R}^k$ that approximates the input vector. Given a set of original inputs $\langle \mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_C \rangle$, the corresponding autoencoder outputs $\langle \hat{\mathbf{s}}_1, \hat{\mathbf{s}}_2, \ldots, \hat{\mathbf{s}}_C \rangle$, the optimization goal is to minimize the difference between the original input $\mathbf{s}$ and the reconstructed $\hat{\mathbf{s}}$ as follows.

$$\arg\min_{\theta_{AE}} \mathcal{L}_{AE} = \arg\min_{\theta_{AE}} \frac{1}{C} \sum_{i=1}^{C} (\mathbf{s}_i - \hat{\mathbf{s}}_i)^2 \tag{1}$$

$$= \arg\min_{\theta_{AE}} \frac{1}{C} \sum_{i=1}^{C} (\mathbf{s}_i - \mathsf{AE}(\mathbf{s}_i))^2, \tag{2}$$

where $\hat{\mathbf{s}}_i = \mathsf{AE}(\mathbf{s}_i)$ and θ_{AE} is the learnable parameters of the AE,

D. Variational Autoencoders (VAEs)

A Variational Autoencoder (VAE) [27] is a directed probabilistic generative model that employs neural networks for learning latent distributions. A VAE consists of an encoder and a decoder. The encoder encodes input data $\mathbf{s}$ into a stochastic latent representation $\mathbf{z}$. The decoder employs the stochastic latent representation $\mathbf{z}$ to generate a reconstructed input data. The posterior probability $p(\mathbf{z}|\mathbf{s})$, which describes the distribution of the encoded, latent variable $\mathbf{z}$ given data $\mathbf{s}$, is implemented by the encoder $q_\phi(\mathbf{z}|\mathbf{s})$, where ϕ includes

the parameters that govern the encoder, e.g., weights for a neural network based encoder. The likelihood $p(\mathbf{s}|\mathbf{z})$ defines the probability of generating the reconstructed input given the distribution of the encoded latent variable $\mathbf{z}$. This is implemented by the decoder $p_\theta(\mathbf{s}|\mathbf{z})$, where θ includes the parameters that govern the decoder. Parameters ϕ and θ are learned by maximizing the likelihood of generating the reconstructed input using a maximum-likelihood objective function [26].

E. Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) are a family of neural networks to model sequential data such as texts [44], paths [48], and time series [12], [13], [21] by capturing sequential dependencies, e.g., temporal dependencies, via the hidden recurrent state $\mathbf{h}$. More specific, the hidden recurrent state $\mathbf{h}_t$ at time step t is a recursive computation based on the hidden state of previous time step $\mathbf{h}_{t-1}$, along with the input at the same time step $\mathbf{s}_t$.

$$\mathbf{h}_t = \mathsf{RNN}(\mathbf{s}_t, \mathbf{h}_{t-1}) \tag{3}$$

A popular form of RNNs is Long-Short Term Memory (LSTM), which improve accuracy by mitigating the gradient vanishing problem when working on long sequences [19]. More specific, an LSTM employs the gating mechanism, including the input gate $\mathbf{i}$, the forget gate $\mathbf{f}$, and the output gate $\mathbf{o}$ to control how much the memory (i.e., information from previous steps) should be kept at each step.

$$\mathbf{i}_t = \sigma(\mathbf{W_i} \cdot \mathbf{s}_t + \mathbf{U_i} \cdot \mathbf{h}_{t-1} + \mathbf{b_i}) \tag{4a}$$
$$\mathbf{f}_t = \sigma(\mathbf{W_f} \cdot \mathbf{s}_t + \mathbf{U_f} \cdot \mathbf{h}_{t-1} + \mathbf{b_f}) \tag{4b}$$
$$\mathbf{o}_t = \sigma(\mathbf{W_o} \cdot \mathbf{s}_t + \mathbf{U_o} \cdot \mathbf{h}_{t-1} + \mathbf{b_o}) \tag{4c}$$
$$\tilde{\mathbf{c}}_t = \mathsf{tanh}(\mathbf{W_c} \cdot \mathbf{s}_t + \mathbf{U_c} \cdot \mathbf{h}_{t-1} + \mathbf{b_c}) \tag{4d}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \tag{4e}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \mathsf{tanh}(\mathbf{c}_t) \tag{4f}$$

Here, $\mathbf{W}$, $\mathbf{U}$, and $\mathbf{b}$ are learnable parameters, $\sigma(\cdot)$ and $\mathsf{tanh}(\cdot)$ are the sigmoid and tanh functions, and $\odot$ denotes the Hamadard product.

A critical limitation of RNNs is that the computational overhead is expensive due to its recursive nature—the hidden state of a time step depends on the hidden state of its previous time step, i.e., $\mathbf{h}_t$ depends on $\mathbf{h}_{t-1}$. This makes it difficult to parallelize the computations. This incurs high computation overhead and may prevent RNNs from time-critical applications where efficient runtime is seriously desirable.

F. Variational RNN Autoencoders (VRAEs)

A Variational Recurrent Neural Network Autoencoder (VRAE) [11], [42] is a recurrent version of VAE for the modeling of sequential data, e.g., time series. At each time step t, VRAE has a VAE with stochastic latent variable $\mathbf{z}_t$, a deterministic, recurrent hidden state $\mathbf{h}_t$ that captures temporal dependency, and observation $\mathbf{s}_t$. VRAE consists of four main steps: modeling the prior distribution, generation, inference, and recurrence (denoted by P, G, I, and R in Fig. 1).

Step P: The latent variable $\mathbf{z}_t$ at time step t is defined by a prior distribution that is conditioned on the previous hidden recurrent state $\mathbf{h}_{t-1}$ (see Fig. 1(a)):

$$\mathbf{z}_t \sim \mathcal{N}(\varphi_\mu^{prior}(\mathbf{h}_{t-1}), \varphi_\sigma^{prior}(\mathbf{h}_{t-1})), \tag{5}$$

where $\mathcal{N}$ is a normal distribution.

Step G: The observation $\mathbf{s}_t$ is generated by conditioning it on the latent variable $\mathbf{z}_t$ and the previous hidden state $\mathbf{h}_{t-1}$ (see Fig. 1(b)). The likelihood of generating $\mathbf{s}_t$ is

$$\mathbf{s}_t|\mathbf{z}_t \sim \mathcal{N}(\varphi_\mu^{dec}(\varphi^{\mathbf{z}}(\mathbf{z}_t), \mathbf{h}_{t-1}), \varphi_\sigma^{dec}(\varphi^{\mathbf{z}}(\mathbf{z}_t), \mathbf{h}_{t-1})) \tag{6}$$

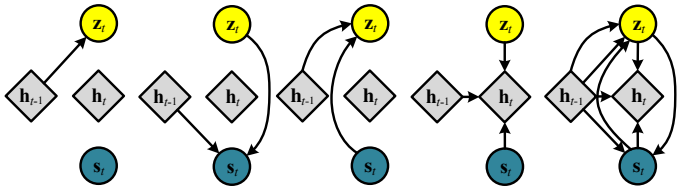
Step I: Similarly, the posterior distribution in the inference step (see Fig. 1(c)) is defined as follows.

$$\mathbf{z}_t|\mathbf{s}_t \sim \mathcal{N}(\varphi_\mu^{enc}(\varphi^{\mathbf{s}}(\mathbf{s}_t), \mathbf{h}_{t-1}), \varphi_\sigma^{enc}(\varphi^{\mathbf{s}}(\mathbf{s}_t), \mathbf{h}_{t-1})) \tag{7}$$

Step R: In the recurrence step, the hidden recurrent state $\mathbf{h}_t$ is obtained from the previous hidden state h_{t-1}, the latent variable $\mathbf{z}_t$, and the data variable $\mathbf{s}_t$ (see Fig. 1(d)).

$$\mathbf{h}_t = f_\theta([\varphi^{\mathbf{s}}(\mathbf{s}_t), \varphi^{\mathbf{z}}(\mathbf{z}_t)], \mathbf{h}_{t-1}) \tag{8}$$

In the above, $\varphi(\cdot)$ represent feed-forward neural networks and $f_\theta(\cdot)$ is a recurrent neural network such as Gated Recurrent Unit (GRU) [10]. Fig. 1(e) shows a VRAE at time step t.



(a) P. (b) G. (c) I. (d) R. (e) Overall.

Fig. 1: VRAE overview. Diamond shapes denote deterministic variables and circles denote stochastic variables.

One design flaw of VRAE is that the deterministic recurrent hidden state $\mathbf{h}$ and the stochastic latent variable $\mathbf{z}$ depend on each other. Specifically, $\mathbf{z}_t$ depends on $\mathbf{h}_{t-1}$ in steps P and I, while $\mathbf{h}_t$ depends on $\mathbf{z}_t$ in step R. The interdependence between the deterministic recurrent hidden states and the stochastic latent variables forms a "lock" that prevents VRAE from extending to a bi-directional formulation, which have been shown to be more powerful than uni-directional models [41]. To illustrate the lock, assume that we have a bi-directional VRAE with deterministic hidden state sequences $\overrightarrow{\mathbf{h}}$ and $\overleftarrow{\mathbf{h}}$ for forward and backward directions, respectively. At timestamp t, $\mathbf{z}_t$ depends on $\overrightarrow{\mathbf{h}}_{t-1}$ in the forward direction and $\overleftarrow{\mathbf{h}}_{t+1}$ in the backward direction; but we cannot have $\overleftarrow{\mathbf{h}}_{t+1}$ because it depends on $\mathbf{z}_{t+1}$, which has not yet been computed. Thus, the flow is locked. Another limitation is that VRAEs use RNNs to learn temporal dependencies, which are inefficient due to the recursive computations.

III. Methodology

We first present the architecture of the Variational Quasi-Recurrent Autoencoder (VQRAE) that enables efficient and parallel computations and supports bidirectional modeling.

Next, we present the objective function that enables robust training which contends with the settings where the training data is contaminated.

A. Framework Overview

We show the overview of the proposed time series anomaly detection framework in Fig. 2. Data Preprocessing is a component that is shared by both offline training and online detection. In the data processing step, we follow an existing procedure [42], [46]. More specifically, time series is standardized and transformed into overlapped subsequences by using a sliding windows of length $B+1$. Then, the preprocessed data, i.e., multiple sequences of length $B+1$, are fed into the model for training. Here, we propose two models, VQRAE and BiVQRAE, which are described in the next sections. After training, the trained models are then used for online anomaly detection. Each sequence is fed into a trained model, and the model produces an anomaly score for each observation in the sequence.

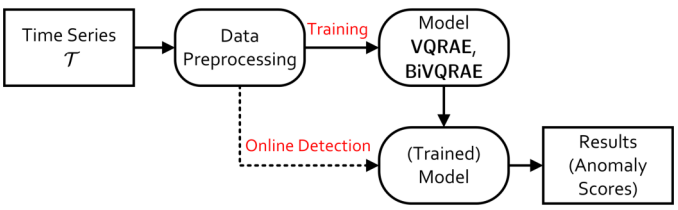


Fig. 2: Framework Overview

B. Network Architecture

VQRAE: An overview of VQRAE is shown in Fig. 3, which consists of an encoder qnet and a decoder pnet. The encoder qnet learns an abstract function $q_\phi(\mathbf{z}_{1:B+1}|\mathbf{s}_{1:B+1})$ that represents the posterior probability of having $B+1$ stochastic hidden states $\mathbf{z}_{1:B+1}$ given a sequence of $B+1$ observations $\mathbf{s}_{1:B+1}$. The decoder pnet learns an abstract function $p_\theta(\mathbf{s}_{1:B+1}|\mathbf{z}_{1:B+1})$ that represents the probabilities of generating the $B+1$ observations given the $B+1$ stochastic hidden states $\mathbf{z}_{1:B+1}$. Here, ϕ and θ are model parameters of the qnet and the pnet, respectively. Inspired by State Space Models [15] and Deep State Space Model [31], we reformulate the encoder qnet to model

$$q_\phi(\mathbf{z}_{1:B+1}|\mathbf{s}_{1:B+1}) = \prod_t q_\phi(\mathbf{z}_t|\mathbf{z}_{t-1}, \mathbf{s}_{1:B+1}), \tag{9}$$

where $\mathbf{z}_t$ is a stochastic hidden state at time t and $\mathbf{s}_{1:B+1}$ is all observations in a time window.

Unlike VRAE that couples the stochastic latent variable and recurrent states, VQRAE decouples them such that: (i) deterministic recurrent states do not depend on stochastic latent variables, while (ii) stochastic latent variables still depend on deterministic recurrent states such that the temporal dependency is captured.

VQRAE introduces two deterministic recurrent states, an $\mathbf{a}$-space and an $\mathbf{h}$-space, which use opposite directions to capture temporal dependencies for the whole window $\mathbf{s}_{1:B+1}$, as shown by the two orange arrows in Fig. 3. First, $\mathbf{h}$-space models the temporal dependency from $\mathbf{s}_1$ to $\mathbf{s}_t$ in a forward direction; while $\mathbf{a}$-space models the temporal dependency from $\mathbf{s}_t$ to $\mathbf{s}_{B+1}$ in a reverse direction. Next, $\mathbf{h}$- and $\mathbf{a}$-spaces are connected such that the temporal dependency of the entire input sequence $\mathbf{s}_{1:B+1}$ is captured, which is then used to generate the stochastic latent variable $\mathbf{z}$. This ensures that $\mathbf{z}$ has the information from the entire input sequence.

More specifically, temporal dependencies modeled by $\mathbf{h}$-space and $\mathbf{a}$-space in VQRAE are captured by Quasi-Recurrent Neural Networks (QRNNs) [6], as shown by orange arrows in Fig. 3. Existing studies often employ RNNs, such as GRUs [10], to capture temporal dependency. However, they are inefficient due to recursive computations. Thus, they are limited to time-critical applications where the runtime is seriously important. In time-critical applications, CNNs are more preferable due to its computations can be ran in parallel. However, CNNs disregard the order of observations in the sequence, where the computation at the current observation does not depend on the computation of previous observations. This limitation reduces the use of CNNs in applications that involve sequential data, such as time series. QRNNs achieve a better trade-off between CNNs and RNNs. Similar to CNNs, QRNNs achieve higher efficiency than RNNs by making most computations independent, which enables higher parallelism (cf. Eqs. 11a–11c). Similar to RNNs, QRNNs respect the dependencies of the overall order of the observations in the sequence by involving information from the previous step, which capture temporal dependencies better than do CNNs (cf. Eq. 11d). We proceed to explain QRNNs in detail.

QRNN: At time step t, a QRNN receives two consecutive observations, e.g., $\mathbf{s}_{t-1}$ and $\mathbf{s}_t$, and computes the current hidden state $\mathbf{h}_t$ using a QRNN unit:

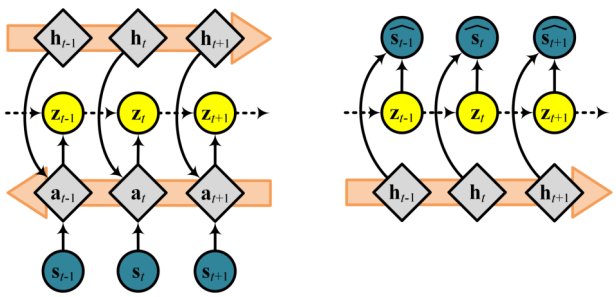
$$\mathbf{h}_t = \mathsf{QRNN}(\mathbf{s}_{t-1}, \mathbf{s}_t) \tag{10}$$

The computation inside a QRNN unit is given as follows.

$$\mathbf{i}_t = \tanh(\mathbf{W}_\mathbf{i}^1 \cdot \mathbf{s}_{t-1} + \mathbf{W}_\mathbf{i}^2 \cdot \mathbf{s}_t + \mathbf{b}_\mathbf{i}) \tag{11a}$$
$$\mathbf{f}_t = \sigma(\mathbf{W}_\mathbf{f}^1 \cdot \mathbf{s}_{t-1} + \mathbf{W}_\mathbf{f}^2 \cdot \mathbf{s}_t + \mathbf{b}_\mathbf{f}) \tag{11b}$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_\mathbf{o}^1 \cdot \mathbf{s}_{t-1} + \mathbf{W}_\mathbf{o}^2 \cdot \mathbf{s}_t + \mathbf{b}_\mathbf{o}) \tag{11c}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + (1 - \mathbf{f}_t) \odot \mathbf{i}_t \tag{11d}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \mathbf{c}_t \tag{11e}$$

Here, $\odot$ denotes element-wise multiplication; $\mathbf{i}_t$, $\mathbf{f}_t$, $\mathbf{o}_t$ are output vectors of the candidate, forget, and output gates, respectively; $\mathbf{c}_t$ is cell state vector; $\mathbf{h}_t$ is a deterministic hidden state vector; $\tanh(\cdot)$ and $\sigma(\cdot)$ are the tanh and sigmoid functions. All $\mathbf{W}$ and $\mathbf{b}$ are learned parameters.

A QRNN computes hidden states using a highly parallelizable procedure with only a minimalist recurrence—the computations of the gates are independent of previous hidden states and thus are parallelizable (cf. Eqs. 11a–11c), and only the cell state vector needs a recursive computation on its previous cell state vector (cf. Eq. 11d). Compared to RNNs where



(a) qnet. (b) pnet.

Fig. 3: VQRAE overview. Orange arrows indicate explicit temporal dependencies of the deterministic variable, computed by QRNN. The dashed arrows indicate implicit temporal dependencies of the stochastic variables. The diamond shape indicates the deterministic variables and the circle shape indicate stochastic variables.

all computations are recursive, QRNNs only involve the one recursive computation in Eq. 11d. This recursion only yields a negligible overhead because it is an element-wise multiplication that can be computed in parallel [6].

Encoder qnet: Recall that an input time series $\mathcal{T}$ is first normalized and then segmented into overlapping sequences by using a sliding window of length $B+1$.

The preprocessed data, i.e., a sequence of length $B+1$, is then fed to qnet as shown in Fig. 3a. At time step t, the deterministic hidden state $\mathbf{h}_t$ is created from the previous observation $\mathbf{s}_{t-1}$ and the current observation $\mathbf{s}_t$ to capture temporal dependencies in the forward direction by using a QRNN layer (see Eq. 12a).

Then, the input observation $\mathbf{s}_t$ is concatenated with the deterministic hidden state $\mathbf{h}_t$. The concatenated variable at time step t and the concatenated variable at time step $t+1$ are sent to another QRNN layer to compute the deterministic hidden state $\mathbf{a}_t$ (see Eq. 12b). This way, $\mathbf{a}_t$ captures temporal dependencies of both directions, i.e., the backward direction through the observations and the forward direction through the $\mathbf{h}$ space. Thus, $\mathbf{a}_t$ captures temporal dependencies of the entire window.

From the deterministic hidden state $\mathbf{a}_t$, we compute the stochastic latent representation $\Phi_{\mathbf{z}_t}$ by using a fully-connected layer (see Eq. 12c). The latent representation $\Phi_{\mathbf{z}_t}$ is fed to two fully-connected layers to generate mean $\mu_{\mathbf{z}_t}$ and standard deviation $\sigma_{\mathbf{z}_t}$ for the stochastic latent variable $\mathbf{z}_t$ (see Eqs. 12d and 12e).

$$\mathbf{h}_t = \mathsf{QRNN}(\mathbf{s}_{t-1}, \mathbf{s}_t) \tag{12a}$$
$$\mathbf{a}_t = \mathsf{QRNN}([\mathbf{s}_{t+1}, \mathbf{h}_{t+1}], [\mathbf{s}_t, \mathbf{h}_t]) \tag{12b}$$
$$\Phi_{\mathbf{z}_t} = f(\mathbf{W}_{\Phi_\mathbf{z}} \cdot \mathbf{a}_t + \mathbf{b}_{\Phi_\mathbf{z}}) \tag{12c}$$
$$\mu_{\mathbf{z}_t} = \mathbf{W}_{\mu_\mathbf{z}} \cdot \Phi_{\mathbf{z}_t} + \mathbf{b}_{\mu_\mathbf{z}} \tag{12d}$$
$$\sigma_{\mathbf{z}_t} = \mathsf{softplus}(\mathbf{W}_{\sigma_\mathbf{z}} \cdot \Phi_{\mathbf{z}_t} + \mathbf{b}_{\sigma_\mathbf{z}}) \tag{12e}$$

Here, $f(\cdot)$ and $\mathsf{softplus}(\cdot)$ are ReLU and softplus functions; $[\cdot,\cdot]$ implies the concatenation of two vectors. All $\mathbf{W}$ and $\mathbf{b}$ are learned parameters. The output of the qnet, $\mathbf{z}_t$, is a diagonal Gaussian sampled from $\mathcal{N}(\mu_{\mathbf{z}_t}, \sigma^2_{\mathbf{z}_t}\mathbf{I})$. To learn a non-Gaussian posterior distribution of $q_\phi(\mathbf{z}_t|\mathbf{s}_t)$, we use Planar Normalizing Flow (PNF) [37] to approximate $\mathbf{z}_t$. PNF is a chain of non-linear invertible mapping that takes $\mathbf{z}_t$ as input: $\mathbf{z}_t^{(K)} = f_{\text{pnf}}^{(K)}(f_{\text{pnf}}^{(K-1)}(...f_{\text{pnf}}^{(1)}(\mathbf{z}_t^{(0)})))$, where $f_{\text{pnf}}^{(k)}, k = \{1 \dots K\}$ is a non-linear invertible mapping function as follows.

$$f_{\text{pnf}}^{(k)}(\mathbf{z}_t^{(k-1)}) = \mathbf{z}_t^{(k-1)} + \mathbf{m} \cdot \mathsf{tanh}(\mathbf{W} \cdot \mathbf{z}_t^{(k-1)} + \mathbf{b}) \tag{13}$$

Here, $\mathbf{m}$, $\mathbf{W}$, and $\mathbf{b}$ are learned parameters. We take $\mathbf{z}_t^{(K)}$, as the stochastic variable $\mathbf{z}_t$ (i.e., $\mathbf{z}_t = \mathbf{z}_t^{(K)}$).

Decoder pnet: In pnet, as shown in Fig. 3b, we reconstruct $\mathbf{s}_t$ from $\mathbf{z}_t$. At time step t, the deterministic hidden state $\mathbf{h}_t$ is concatenated with the stochastic latent variable $\mathbf{z}_t$. Then, the concatenated variable is fed to a fully connect layer to compute the $\Phi_{\mathbf{s}_t}$ (see Eq. 14a). In particular, $\Phi_{\mathbf{s}_t}$ is the learned stochastic representation in the decoder at time step t, which is used to reconstruct $\mathbf{s}_t$. The original representation $\Phi_{\mathbf{s}_t}$ is fed to two fully-connected layers to generate mean $\mu_{\mathbf{s}_t}$ and standard deviation $\sigma_{\mathbf{s}_t}$ for the original variable $\mathbf{s}_t$ (see Eqs. 14b and 14c). The reconstructed data $\hat{\mathbf{s}}_t$ is sampled from $\mathcal{N}(\mu_{\mathbf{s}_t}, \sigma^2_{\mathbf{s}_t}\mathbf{I})$. If $\mathbf{s}_t$ is an anomaly, $\hat{\mathbf{s}}_t$ is significantly different from $\mathbf{s}_t$. Thus, we detect anomalies by using the reconstruction probability of $\hat{\mathbf{s}}_t$. Finally, pnet is formulated as follows.

$$\Phi_{\mathbf{s}_t} = f(\mathbf{W}_{\Phi_\mathbf{s}} \cdot [\mathbf{h}_t, \mathbf{z}_t] + \mathbf{b}_{\Phi_\mathbf{s}}) \tag{14a}$$
$$\mu_{\mathbf{s}_t} = \mathbf{W}_{\mu_\mathbf{s}} \cdot \Phi_{\mathbf{s}_t} + \mathbf{b}_{\mu_\mathbf{s}} \tag{14b}$$
$$\sigma_{\mathbf{s}_t} = \mathsf{softplus}(\mathbf{W}_{\sigma_\mathbf{s}} \cdot \Phi_{\mathbf{s}_t} + \mathbf{b}_{\sigma_\mathbf{s}}) \tag{14c}$$

Here, $f(\cdot)$ and $\mathsf{softplus}(\cdot)$ are the ReLU and softplus functions. All $\mathbf{W}$ and $\mathbf{b}$ are learned parameters.

C. Objective Function

We train qnet and pnet jointly by using Stochastic Gradient Variational Bayes (SGVB), and the parameters ϕ and θ are learned by maximizing the Evidence of Lower BOund (ELBO) [27]. The objective function of VQRAE at each time step t, $\mathcal{L}(\mathbf{s}_t)$, then is derived as follows.

$$\begin{aligned}\arg\max_{\phi,\theta} \mathcal{L}(\mathbf{s}_t) = \mathbb{E}_{q_\phi(\mathbf{z}_t|\mathbf{s}_t)}[\log p_\theta(\mathbf{s}_t|\mathbf{z}_t)] \\ - \mathsf{D}_{\mathsf{KL}}[q_\phi(\mathbf{z}_t|\mathbf{s}_t)||p_\theta(\mathbf{z}_t)]\end{aligned} \tag{15}$$

The first term is a *log-likelihood* that measures the likelihood of the reconstructed data being generated by the decoder, which is returned by the pnet (see Eq. 15). The second term computes the *Kullback-Leibler (KL) divergence* that measures the difference between two distributions–the encoder distribution $q_\phi(\mathbf{z}_t|\mathbf{s}_t)$, which is returned by the qnet (see Eqs. 12d and 12e), and the latent prior distribution $p_\theta(\mathbf{z}_t)$, which is assumed to be a multivariate Gaussian distribution $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

This objective function is not robust to anomalies. Since we consider unsupervised anomaly detection, the training data may not be fully clean, i.e., already including anomalies. Then, when using the first reconstruction term, the objective function takes into account the likelihood of generating such anomalies,

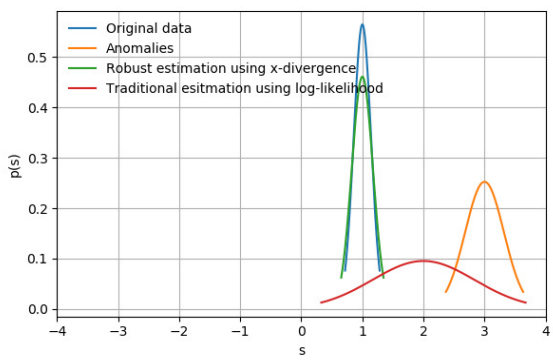


Fig. 4: Example of robust vs. traditional estimations when training data are with anomalies

which should instead be ignored. Fig. 4 shows robust estimation (i.e., robust objective function) vs. non-robust estimation (i.e., non-robust objective function).

To make VQRAE robust to anomalies, we modify the objective function by replacing the first term in Eq. 15, which is not robust to anomalies, with α, β, and γ-divergence [3], [17] that are more robust to anomalies. The updated, robust objective function is show in Eq. 16.

Coming from robust statistics, α, β, and γ-divergence are metrics that compute the distance between two density functions (i.e., distributions). With α, β, and γ-divergence, we weight each observation according to its density from the decoder; if an observation is an anomaly, it often has a low density, and it is thus down weighted [39].

$$\begin{aligned}\arg\max_{\phi,\theta} \mathcal{L}(\mathbf{s}_t) = & -\mathbb{E}_{q_\phi(\mathbf{z}_t|\mathbf{s}_t)}[\mathsf{D}_{\alpha,\beta,\gamma}(\hat{p}(\mathbf{s}_t)||p_\theta(\mathbf{s}_t|\mathbf{z}_t))] \\ & - \mathsf{D}_{\mathsf{KL}}[q_\phi(\mathbf{z}_t|\mathbf{s}_t)||p_\theta(\mathbf{z}_t)],\end{aligned} \tag{16}$$

where $\hat{p}(\mathbf{s}_t)$ denotes the empirical distribution of observations $\mathbf{s}_t$ (i.e., the input data), and $\mathsf{D}_{\alpha,\beta,\gamma}$ $(\hat{p}(\mathbf{s}_t)||p_\theta(\mathbf{s}_t|\mathbf{z}_t))$ are α, β, and γ-divergence, which are formulated as follows.

$$\mathsf{D}_\alpha(\hat{p}(\mathbf{s}_t)||p_\theta(\mathbf{s}_t|\mathbf{z}_t)) = \frac{1}{\alpha-1}\log\int \hat{p}(\mathbf{s}_t)^\alpha p_\theta(\mathbf{s}_t|\mathbf{z}_t)^{1-\alpha} d\mathbf{s}_t \tag{17}$$

$$\begin{aligned}\mathsf{D}_\beta(\hat{p}(\mathbf{s}_t)||p_\theta(\mathbf{s}_t|\mathbf{z}_t)) = & -\frac{\beta+1}{\beta}\int \hat{p}(\mathbf{s}_t) p_\theta(\mathbf{s}_t|\mathbf{z}_t)^\beta d\mathbf{s}_t \\ & + \frac{1}{\beta}\int \hat{p}(\mathbf{s}_t)^{\beta+1} d\mathbf{s}_t + \int p_\theta(\mathbf{s}_t|\mathbf{z}_t)^{\beta+1} d\mathbf{s}_t\end{aligned} \tag{18}$$

$$\begin{aligned}\mathsf{D}_\gamma(\hat{p}(\mathbf{s}_t)||p_\theta(\mathbf{s}_t|\mathbf{z}_t)) = & -\frac{1}{\gamma}\log\int \hat{p}(\mathbf{s}_t) p_\theta(\mathbf{s}_t|\mathbf{z}_t)^\gamma d\mathbf{s}_t \\ & + \frac{1}{\gamma(\gamma+1)}\log\int \hat{p}(\mathbf{s}_t)^{\gamma+1} d\mathbf{s}_t + \frac{1}{\gamma+1}\log\int p_\theta(\mathbf{s}_t|\mathbf{z}_t)^{\gamma+1} d\mathbf{s}_t\end{aligned} \tag{19}$$

The empirical distribution $\hat{p}(\mathbf{s}_t)$ is difficult to estimate in practice. To avoid estimating $\hat{p}(\mathbf{s}_t)$, we reformulate the robust objective functions with the α, β, and γ-divergence as equivalent robust objective functions using weighted mean square errors between the original observation $\mathbf{s}_t$ and the reconstructed observation $\hat{\mathbf{s}}_t$, which are much easier to compute, and the weights are defined differently [2], [17]. More specifically, the α, β, and γ-divergence (cf. Eqs. 17, 18, and 19) are equivalent to the following:

$$\mathsf{D}_\alpha(\hat{p}(\mathbf{s}_t)||p_\theta(\mathbf{s}_t|\mathbf{z}_t)) = \frac{1}{\alpha-1}\log\left(\frac{1}{(2\pi\sigma^2)^{\frac{\alpha}{2}}}\exp\left(-\frac{1}{2\sigma^2}||\mathbf{s}_t-\hat{\mathbf{s}}_t||^2\right)^{1-\alpha}\right) \tag{20}$$

$$\mathsf{D}_\beta(\hat{p}(\mathbf{s}_t)||p_\theta(\mathbf{s}_t|\mathbf{z}_t)) = -\frac{\beta+1}{\beta}\left(\frac{1}{(2\pi\sigma^2)^{\frac{\beta}{2}}}\exp\left(-\frac{1}{2\sigma^2}||\mathbf{s}_t-\hat{\mathbf{s}}_t||^2\right)^{\beta} - 1\right) \tag{21}$$

$$\mathsf{D}_\gamma(\hat{p}(\mathbf{s}_t)||p_\theta(\mathbf{s}_t|\mathbf{z}_t)) = \frac{\gamma+1}{\gamma}\left(\frac{1}{(2\pi\sigma^2)^{\frac{\gamma}{2}}}\exp\left(-\frac{1}{2\sigma^2}||\mathbf{s}_t-\hat{\mathbf{s}}_t||^2\right)^{\gamma}\right) \tag{22}$$

In practice, σ does not affect the optimization of the robust objective function. For simplicity, we set σ to 1. The weighted mean square errors maintain the property of the α, β, and γ-divergence. More specifically, the weighted mean square errors can suppress the contributions of anomalies in the objective function, thus reducing the chances of contamination by anomalies.

Because each input sequence is overlapping subsequences created by a sliding window of length of $B+1$ [42], an objective function is computed on each input sequence $\mathbf{s}_{t-B:t}$.

$$\begin{aligned}&\arg\max_{\phi,\theta} \mathcal{L}(\mathbf{s}_{t-B:t}) = \\ &-\mathbb{E}_{q_\phi(\mathbf{z}_{t-B:t}|\mathbf{s}_{t-B:t})}[\mathsf{D}_{\alpha,\beta,\gamma}(\hat{p}(\mathbf{s}_{t-B:t})||p_\theta(\mathbf{s}_{t-B:t}|\mathbf{z}_{t-B:t}))] \\ &-\mathsf{D}_{\mathsf{KL}}[q_\phi(\mathbf{z}_{t-B:t}|\mathbf{s}_{t-B:t})||p_\theta(\mathbf{z}_{t-B:t})]\end{aligned} \tag{23}$$

Monte Carlo integration [38] is used to approximate the expectation in Eq. 23, where $\mathbf{z}_{t-B:t}^{(l)}, l = 1, \ldots, L$ are samples from $q_\phi(\mathbf{z}_{t-B:t}|\mathbf{s}_{t-B:t})$.

$$\begin{aligned}\mathcal{L}(\mathbf{s}_{t-B:t}) \approx \frac{1}{L}\sum_{l=1}^{L} & -[\mathsf{D}_{\alpha,\beta,\gamma}(\hat{p}(\mathbf{s}_{t-B:t})||p_\theta(\mathbf{s}_{t-B:t}|\mathbf{z}_{t-B:t}^{(l)}))] \\ & - \mathsf{D}_{\mathsf{KL}}[q_\phi(\mathbf{z}_{t-B:t}^{(l)}|\mathbf{s}_{t-B:t})||p_\theta(\mathbf{z}_{t-B:t}^{(l)})].\end{aligned} \tag{24}$$

To resolve the KL-vanishing problem [5], which reduces the model to a deterministic function, we adopt the KL-annealing trick [5] in the implementation. More specifically, we use a hyperparameter to control the magnitude of the KL-divergence. When the training starts, the hyperparameter is close to zero, such that the model learns to encode as much information into stochastic representation $\mathbf{z}$ as possible and to ignore the distribution of $\mathbf{z}$. Then, the hyperparameter increases gradually, forcing the model to follow the prior.

D. Bi-directional Architecture

Existing methods on VRAE [11], [16] only consider uni-directional computations. At a specific time step, the computation only considers observations that are earlier than the time step. Recall that the input is a sequence of size $B+1$, and thus the observations after the time step are also available. However, uni-directional model fails to capture observations that after the time step.

This is because VRAE couples the stochastic hidden state with deterministic hidden states that capture temporal dependencies, while VQRAE decouples them. When coupled, the flow

is locked. As an example, assume that we have a bi-directional VRAE with deterministic hidden state sequences $\overrightarrow{\mathbf{h}}$ and $\overleftarrow{\mathbf{h}}$ for forward and backward directions, respectively. In the P step of the forward pass, $\mathbf{z}_1$ depends on $\overrightarrow{\mathbf{h}}_0$ and $\overleftarrow{\mathbf{h}}_2$; but we cannot have $\overleftarrow{\mathbf{h}}_2$ because it depends on $\mathbf{z}_2$, which has not yet been computed. Similarly, in the P step of backward pass, $\mathbf{z}_{B+1}$ depends on $\overrightarrow{\mathbf{h}}_B$ and $\overleftarrow{\mathbf{h}}_{B+2}$; but we cannot have $\overrightarrow{\mathbf{h}}_B$ because it depends on $\mathbf{z}_B$, which has not yet been computed. Thus, the flow is locked, making VRAE fail to support bi-directional modeling. In VQRAE, the deterministic states do not depend on the stochastic states. Thus, the flows are not locked, making bi-directional modeling possible.

We extend VQRAE to bi-directional VQRAE (BiVQRAE) to further improve the performance by capturing information from the observations from both before and after a specific time step. An overview of BiVQRAE is shown in Fig. 5. BiVQRAE consists of two components—biqnet and bipnet, equivalent to the encoder and the decoder functionalities in VQRAE, respectively.

Because the input for the model is a sequence of length $B+1$, biqnet learns an abstract function $q_\phi(\mathbf{z}_{1:B+1}|\mathbf{s}_{1:B+1})$ and bipnet learns an abstract function $p_\theta(\mathbf{s}_{1:B+1}|\mathbf{z}_{1:B+1})$. Our goal is to model

$$q_\phi(\mathbf{z}_{1:B+1}|\mathbf{s}_{1:B+1}) = \prod_t q_\phi(\mathbf{z}_t|\mathbf{z}_{t-1}, \mathbf{z}_{t+1}, \mathbf{s}_{1:B+1}). \tag{25}$$

The difference between VQRAE and BiVQRAE is the factorization of $\mathbf{z}_t$ into $\mathbf{z}_{t-1}$ and $\mathbf{z}_{t+1}$ (see Eq. 25) by using bi-directional computation for deterministic hidden states (i.e., $\mathbf{h}_t$ and $\mathbf{a}_t$) in BiVQRAE. Specifically, variable $\mathbf{h}_t$ in VQRAE is represented by forward and backward variables $\overrightarrow{\mathbf{h}}_t$ and $\overleftarrow{\mathbf{h}}_t$ in BiVQRAE. Similarly, variable $\mathbf{a}_t$ is represented by $\overrightarrow{\mathbf{a}}_t$ and $\overleftarrow{\mathbf{a}}_t$.

In particular, the biqnet is formulated as follows.

$$\overrightarrow{\mathbf{h}}_t = \mathsf{QRNN}(\mathbf{s}_{t-1}, \mathbf{s}_t) \tag{26a}$$
$$\overleftarrow{\mathbf{h}}_t = \mathsf{QRNN}(\mathbf{s}_t, \mathbf{s}_{t+1}) \tag{26b}$$
$$\overrightarrow{\mathbf{a}}_t = \mathsf{QRNN}([\mathbf{s}_{t-1}, \overleftarrow{\mathbf{h}}_{t-1}], [\mathbf{s}_t, \overleftarrow{\mathbf{h}}_t]) \tag{26c}$$
$$\overleftarrow{\mathbf{a}}_t = \mathsf{QRNN}([\mathbf{s}_{t+1}, \overrightarrow{\mathbf{h}}_{t+1}], [\mathbf{s}_t, \overrightarrow{\mathbf{h}}_t]) \tag{26d}$$
$$\Phi_{\mathbf{z}_t} = f(\mathbf{W}_{\Phi_\mathbf{z}} \cdot [\overrightarrow{\mathbf{a}}_t, \overleftarrow{\mathbf{a}}_t] + \mathbf{b}_{\Phi_\mathbf{z}}) \tag{26e}$$
$$\mu_{\mathbf{z}_t} = \mathbf{W}_{\mu_\mathbf{z}} \cdot \Phi_{\mathbf{z}_t} + \mathbf{b}_{\mu_\mathbf{z}} \tag{26f}$$
$$\sigma_{\mathbf{z}_t} = \mathsf{softplus}(\mathbf{W}_{\sigma_\mathbf{z}} \cdot \Phi_{\mathbf{z}_t} + \mathbf{b}_{\sigma_\mathbf{z}}) \tag{26g}$$

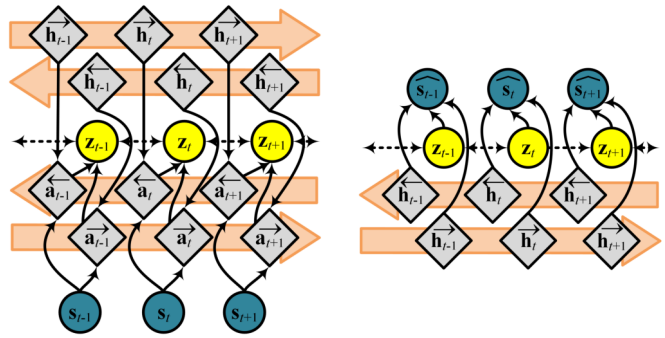
The bipnet is formulated as follows.

$$\Phi_{\mathbf{s}_t} = f(\mathbf{W}_{\Phi_\mathbf{s}} \cdot [\overrightarrow{\mathbf{h}}_t, \overleftarrow{\mathbf{h}}_t, \mathbf{z}_t] + \mathbf{b}_{\Phi_\mathbf{s}}) \tag{27a}$$
$$\mu_{\mathbf{s}_t} = \mathbf{W}_{\mu_\mathbf{s}} \cdot \Phi_{\mathbf{s}_t} + \mathbf{b}_{\mu_\mathbf{s}} \tag{27b}$$
$$\sigma_{\mathbf{s}_t} = \mathsf{softplus}(\mathbf{W}_{\sigma_\mathbf{s}} \cdot \Phi_{\mathbf{s}_t} + \mathbf{b}_{\sigma_\mathbf{s}}) \tag{27c}$$

E. Anomaly Scores

We define an anomaly score of $\mathbf{s}_t$ using the conventional negative log-likelihood,

$$\mathcal{AS}(\mathbf{s}_t) = -\mathbb{E}_{q_\phi(\mathbf{z}_t|\mathbf{s}_t)}[\hat{p}(\mathbf{s}_t)||p_\theta(\mathbf{s}_t|\mathbf{z}_t)] = ||\mathbf{s}_t - \hat{\mathbf{s}}_t||^2 \tag{28}$$



(a) biqnet. (b) bipnet.

Fig. 5: BiVQRAE overview. The orange arrows indicate explicit temporal dependencies between deterministic variables,which are computed by QRNN. The dash arrows indicate implicit temporal dependencies between stochastic variables.

A high score indicates that the input $\mathbf{s}_t$ is reconstructed well. A low score indicates that the input $\mathbf{s}_t$ is reconstructed poorly and is more likely to be anomalous.

As we use overlapping subsequences, an observation may appear in multiple subsequences and thus have multiple anomaly scores. To solve this problem, we consider anomaly scores for all observations for the first subsequence. For each of the remaining subsequences, we consider only the anomaly score of the last timestamp in the subsequence. We do so to support better anomaly detection in streaming settings. In streaming settings, once we receive a new observation, we use the most recent B observations, which can be stored in a buffer, to create a new window. We can then process this window using the proposed models and use the anomaly score of the last timestamp in the window as the anomaly score for the newly arrived observation.

IV. Experimental Study

A. Experimental Setup

Datasets: We use five real-world time series datasets to evaluate our methods: (1) Genesis Data (*GD*)[1] contains two 18-dimensional time series and three 24-dimensional time series. All time series are collected from pick-and-place robots. Each time series contains from ca. 6,000 to 16,000 observations. (2) High Storage System (*HSS*)[2] contains four 18-dimensional time series that are collected from a high storage system. Each time series contains from ca. 19,000 to 25,000 observations. (3) Electrocardiogram (*ECG*)[3] contains 2-dimensional electrocardiogram time sequences collected from seven patients, each with 3,750 to 5,400 observations. (4) Trajectory Data (*TD*)[3] contains seven sets of 2-dimensional time series that are converted from trajectories. Each time series contains approximately 1,000 observations. (5) Yahoo S5! (*S5*)[4]

[1] https://www.kaggle.com/inIT-OWL/genesis-demonstrator-data-for-machine-learning

[2] https://www.kaggle.com/inIT-OWL/high-storage-system-data-for-energy-optimization

[3] https://www.cs.ucr.edu/∼eamonn/discords/

[4] https://webscope.sandbox.yahoo.com/

TABLE II: Datasets overview.

	Avg. length	Avg. #anomalies	Anomaly ratio (%)	Channels	#Time series
GD	9,076	50	0.8	18	5
HSS	19,999	1,669	16.6	18	4
ECG	4,457	217	4.8	2	7
TD	1,000	392	39.2	2	76
S5	1,419	13	0.9	1	163

includes a synthetic and a real-world dataset representing the workloads of different Yahoo services. Each dataset has around 100 time series, each with ca. 1,400 observations;

Table II summarizes all datasets in the experiment.

All datasets come with ground truth anomaly labels. We do not use these labels during training. We only use them for testing, i.e., for evaluating accuracy. In addition, we train all models using time series with anomalies, i.e., contaminated data, not clean data, to study different algorithms robustness to anomalies.

Baselines: We compare our methods with ten existing approaches: (1) Gaussian Mixture Model Variational Autoencoder (GMMVAE) [29], a variational autoencoder on top of a Gaussian mixture distribution; (2) Randomized Neural Network (RN) [8], an ensemble autoencoders for anomaly detection; (3) CNN-based Autoencoder (CAE) [25], which treats time series as images and employs a CNN-based autoencoder to reconstruct images; (4) RNN-based Autoencoder (RAE) [32], a recurrent autoencoder using GRU to reconstruct time series via a sequence-to-sequence architecture; (5) BeatGAN (BGAN) [54], a recurrent autoencoder using GRU to reconstruct time series via a sequence-to-sequence architecture; (6) RNN-based Variational Autoencoder (RNNVAE) [34], a hybrid model, which stacks a VAE on top of GRU at each time step to reconstruct time series; (7) Transformer Autoencoder (TAE) [33], a transformer-based autoencoder using attention mechanism to reconstruct time series; (8) Multi-scale CNN-RNN based Autoencoder (MSCRAE) [51], a multi-scale hybrid model, which combines CNN and RNN to reconstruct the time series; (9) Variational Autoencoder (VAE) [46], a stochastic method, that uses VAE to learn latent variables of time series; (10) Variational RNN Autoencoder (VRAE) [42], a sequential stochastic method, that uses VRAE to learn latent variables of time series.

Evaluation Metrics: An observation $\mathbf{s}_i$ is considered as an anomaly if $\mathcal{AS}(\mathbf{s}_i)$ exceeds a threshold τ; otherwise, it is a clean observation. However, how to choose a proper threshold τ is non-trivial, and it requires prior knowledge or efforts from domain experts. Following an existing strategy [8], we consider all possible thresholds by employing two metrics—Area Under the Curve of Precision-Recall (*PR-AUC*) [40] and Area Under the Curve of Receiver Operating Characteristic (*ROC-AUC*) [40]. They are defined based on true positives (*TP*) (i.e., anomalies predicted as anomalies), true negatives (*TN*) (i.e., inliers predicted as inliers), false positives (*FP*) (i.e., inliers predicted as anomalies), and false negatives (*FN*) (i.e., anomalies predicted as inliers). The only difference is that *PR-AUC* does not take *TN* into account, while *ROC-AUC* does.

Both metrics reflect the accuracy and robustness of an anomaly detection method without the need for choosing any specific threshold. The higher the *PR-AUC* and *ROC-AUC*, the more accuracy and more robust a method is.

Implementation Details: We implement GMMVAE, RN, CAE, RAE, BGAN, TAE, MSCRAE, RNNVAE, VAE, VRAE VQRAE, and BiVQRAE, using PyTorch 1.1.0 on Python 3.7. The source code is available at `https://github.com/tungk/Bi-VQRAE`. Experiments are conducted on a Linux workstation with 64 AMD CPUs with 512 GB RAM and 2 NVIDIA Titan V GPUs with 24 GB RAM.

Hyperparameter Settings: Since we study unsupervised anomaly detection, we are unable to tune the hyperparamters using labeled data. To ensure fair comparisons, we consider different hyperparameter settings, and report the median result for all methods. We do not report the best result because in unsupervised settings, we have no labeled data to enable identifying the best hyperparameters that lead to the best result. Specifically, we define a range for each hyperparameter. We then use random search with random 200 combinations to explore the hyperparameter space and identify a hyperparameter setting that gives the median result among all the explored hyperparameter settings. We then report this median result and consider this hyperparameter setting as the default setting. Next, we conduct experiments to study the sensitivity of different hyperparameters. To do so, each time, we vary a chosen hyperparameter in its range while fixing the other hyperparameters to their default settings.

We proceed to provide the ranges for the hyperparameters. We vary α, β, and γ among 0.0001, 0.001, 0.01, 0.1, 1, and 10. We vary window size B among 8, 16, 32, 64, 128, and 256. For all the neural network based methods, we vary the number of feed-forward layers (GMMVAE, RN, TAE, and VAE) and the number of CNNs layers (CAE, BGAN, and MSCRAE) among 3, 5, 7, 9, and 11; next, we vary the number of kernels in each layer (CAE, BGAN, and MSCRAE), the number of hidden units ($\mathbf{h}$ and $\mathbf{a}$-space) (GMMVAE, RN, RAE, RNNVAE, MSCRAE, VAE, VRAE, VQRAE, and BiVQRAE), and the stochastic latent variable ($\mathbf{z}$-space) size (GMMVAE, RNNVAE, VAE, and VRAE, VQRAE, and BiVQRAE) among 1, 2, 3, 4, 8, 16, 32, 64, 128, and 256; next, we vary the kernel size (CAE, BGAN, and MSCRAE) among 3, 5, 7, 9, and 11. For GMMVAE, we vary the number of Gaussian components among 1, 2, 4, 8, 16, and 32. For RN, we vary the number of base models among 5, 10, 20, 50, 100, and 500. For TAE, we vary the number of attention heads among 3, 5, 7, 9, and 11. The Monte Carlo sampling hyperparameter L (cf. Eq. 24) is set to 5. We also study the effect of $\mathbf{h}$, $\mathbf{a}$, and $\mathbf{z}$-dimension and window size B, showing that they are insensitive to the performance of the models. For RNNAE, MSCREA, RNNVAE, and VRAE, we implement two RNNs layers to learn temporal dependencies. For VQRAE, we implement two QRNNs layers to learn $\mathbf{h}_t$ and two QRNNs layers to learn $\mathbf{a}_t$ (see Eqs. 12a and 12b). For BiVQRAE, we implement two QRNNs layers to learn $\overrightarrow{\mathbf{h}}_t$, $\overleftarrow{\mathbf{h}}_t$, $\overrightarrow{\mathbf{a}}_t$, and $\overleftarrow{\mathbf{a}}_t$, respectively (see Eqs. 26a, 26b, 26c, and 26d). Using the Planar Normalizing Flow component, we implement ten layers for the non-linear

TABLE III: Overall accuracy, *PR-AUC*.

	GMMVAE	RN	CAE	RAE	BGAN	TAE	MSCREA	RNNVAE	VAE	VRAE	VQRAE			BiVQRAE		
											α	β	γ	α	β	γ
GD	0.142	0.112	0.116	0.128	0.109	0.088	0.075	0.086	0.097	0.131	0.235	0.242	0.245	0.249	0.256	**0.258**
HSS	0.216	0.146	0.207	0.242	0.214	0.195	0.161	0.204	0.203	0.219	0.225	0.223	0.222	**0.227**	0.224	0.222
ECG	0.163	0.105	0.107	0.118	0.103	0.138	0.105	0.079	0.131	0.144	0.177	0.177	0.184	0.141	**0.189**	0.186
TD	0.364	0.168	0.177	0.163	0.151	0.175	0.148	0.118	0.188	0.165	0.428	0.427	0.423	0.429	0.430	**0.432**
S5	0.458	0.232	0.383	0.421	0.434	0.298	N/A	0.211	0.272	0.298	0.487	0.525	0.499	0.490	**0.527**	0.524
Wins	0	0	0	0	0	0	0	0	0	0	0	0	0	1	**2**	**2**

TABLE IV: Overall accuracy, *ROC-AUC*.

	GMMVAE	RN	CAE	RAE	BGAN	TAE	MSCREA	RNNVAE	VAE	VRAE	VQRAE			BiVQRAE		
											α	β	γ	α	β	γ
GD	0.763	0.731	0.641	0.608	0.673	0.652	0.582	0.595	0.664	0.658	0.970	0.968	0.969	0.975	0.976	**0.978**
HSS	0.534	0.526	0.560	0.537	0.549	**0.563**	0.509	0.516	0.525	0.521	0.529	0.520	0.524	0.538	0.527	0.526
ECG	0.533	0.524	0.574	0.552	0.547	0.542	0.509	0.536	0.531	0.551	0.592	0.583	0.598	0.597	**0.603**	0.601
TD	0.531	0.533	0.583	0.593	0.622	0.531	0.519	0.574	0.643	**0.662**	0.539	0.535	0.547	0.542	0.546	0.549
S5	0.815	0.575	0.757	0.753	0.677	0.635	N/A	0.642	0.678	0.660	0.858	0.849	0.875	0.873	0.864	**0.882**
Wins	0	0	0	0	0	1	0	0	0	1	0	0	0	0	1	**2**

invertible mapping function (i.e., $K = 10$, see Eq. 13). For all methods, we set the batch size to 64 and the learning rate to 0.001, and we use the Adam optimizer.

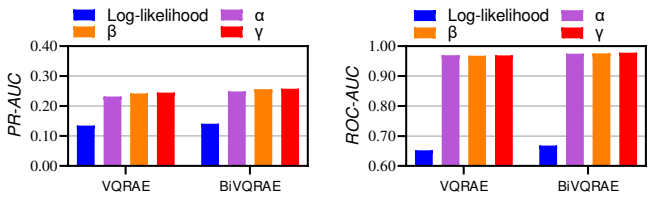
B. Experimental Results

Anomaly Detection Performance: Tables III and IV show that both our methods, VQRAE and BiVQRAE, achieve on average the best accuracy. In addition, BiVQRAE performs the best on all datasets w.r.t. *PR-AUC* and performs the best on three datasets w.r.t. *ROC-AUC* and outperforms VQRAE in all cases. This indicates that the bi-directional architecture outperforms single directional architecture, which conforms that bidirectional contribute to improved accuracy as reported in the literature. Note that due to the interdependence issue in VRAE, it is impossible to enable a bi-directional architecture with VRAE. The methods that take into account the temporal dependence inside the architecture (e.g., VRAE) outperform the methods that do not do so (e.g., VAE and RN). The stochastic methods that learn latent distributions (e.g., VAE) outperform the non-stochastic methods that disregard latent distributions (e.g., RN). The methods that take into account the robustness (e.g., VQRAE and BiVQRAE) outperform the methods that do not do so.

Effect of Objective Functions and Robustness: We study the effect of using traditional log-likelihood for reconstruction objective functions (see Eq. 15). We compare the performance of the model when using log-likelihood vs. α, β, and γ-divergence. We only report on the effect of objective functions on dataset *GD*. The results on the other datasets show similar trends. We show the results on *GD* in Fig. 6.

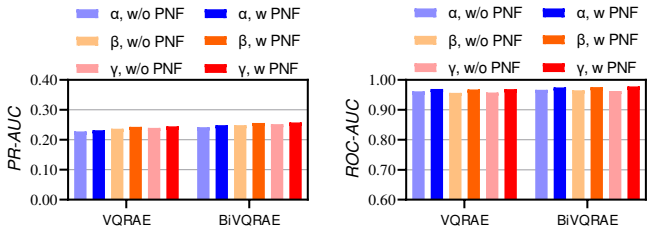
The results demonstrate that the models that use α, β, and γ-divergence outperform traditional log-likelihood approaches w.r.t. *PR-AUC* and *ROC-AUC*. This indicates that using α, β, and γ-divergence make VQRAE and BiVQRAE more robust and fits better the unsupervised setting for anomaly detection, where the training data itself may be contaminated.

Effect of using Planar Normalizing Flow (PNF): We study the effect of using PNF. We compare the performance of the model when using PNF and without using PNF. We only report on the effect of using PNF on dataset *GD*. The results for the other datasets show similar trends. Fig. 7 shows the results w.r.t. *PR-AUC* and *ROC-AUC*



(a) *PR-AUC*. (b) *ROC-AUC*.

Fig. 6: Effect of objective functions, *GD*, *PR-AUC* and *ROC-AUC*.



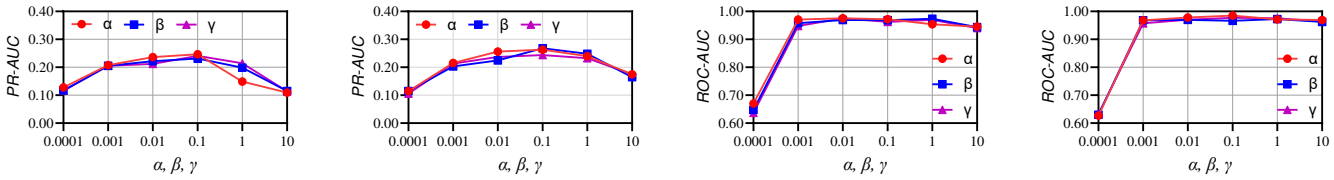
(a) *PR-AUC*. (b) *ROC-AUC*.

Fig. 7: Effect of using PNF, *PR-AUC* and *ROC-AUC*.

The results demonstrate that the models that use PNF are better than the models that do not use PNF w.r.t. *PR-AUC* and *ROC-AUC*. This indicates that PNF helps VQRAE and BiVQRAE improving the accuracy.

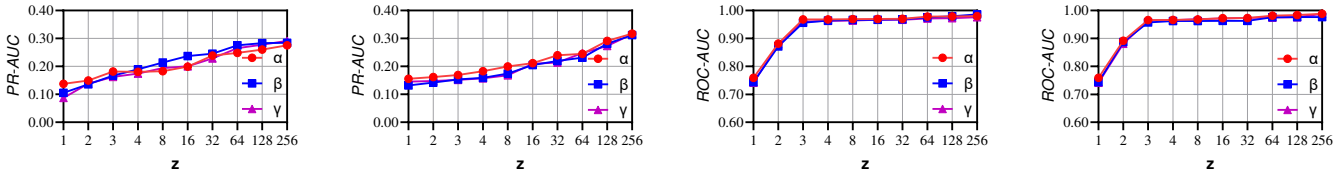
Effect of α, β, and γ: We vary α, β, and γ among 0.0001, 0.001, 0.01, 0.1, 1, and 10. Intuitively, a small α, β, and γ means that most data is normal; thus, the model is not robust to anomalies. In contrast, a large α, β, and γ encourages more observations to be considered as anomalies. We only report on the effect of α, β, and γ on dataset *GD*. The results for the other datasets show similar trends. Fig. 8 shows the results w.r.t. *PR-AUC* and *ROC-AUC*.

All methods reach the best performance when α, β, and γ are between 0.01 and 0.1, indicating that α, β, and γ in the range [0.01, 0.1] are the most suitable setting for our datasets. In real-world settings, choosing α, β, and γ may be done using



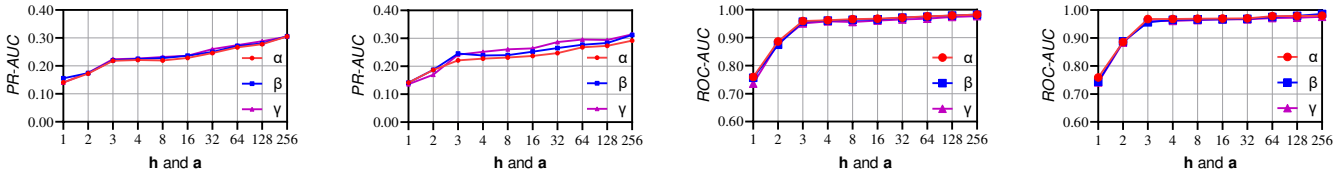
(a) *GD*, VQRAE, *PR-AUC*. (b) *GD*, BiVQRAE, *PR-AUC*. (c) *GD*, VQRAE, *ROC-AUC*. (d) *GD*, BiVQRAE, *ROC-AUC*.

Fig. 8: Effect of α, β, and γ, *PR-AUC* and *ROC-AUC*.



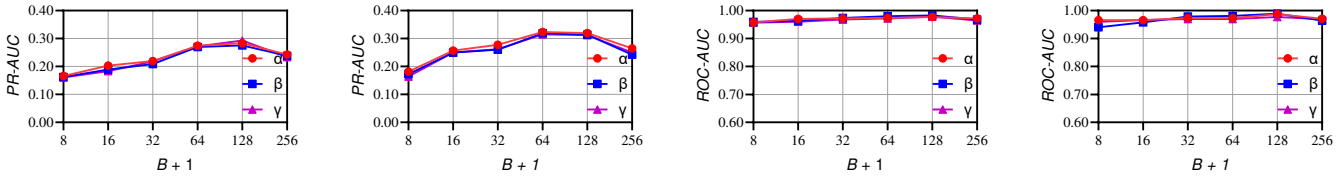
(a) *GD*, VQRAE, *PR-AUC*. (b) *GD*, BiVQRAE, *PR-AUC*. (c) *GD*, VQRAE, *ROC-AUC*. (d) *GD*, BiVQRAE, *ROC-AUC*.

Fig. 9: Effect of different $\mathbf{z}$-space dimension, *PR-AUC* and *ROC-AUC*.



(a) *GD*, VQRAE, *PR-AUC*. (b) *GD*, BiVQRAE, *PR-AUC*. (c) *GD*, VQRAE, *ROC-AUC*. (d) *GD*, BiVQRAE, *ROC-AUC*.

Fig. 10: Effect of different $\mathbf{h}$ and $\mathbf{a}$-space dimension, *PR-AUC* and *ROC-AUC*.



(a) *GD*, VQRAE, *PR-AUC*. (b) *GD*, BiVQRAE, *PR-AUC*. (c) *GD*, VQRAE, *ROC-AUC*. (d) *GD*, BiVQRAE, *ROC-AUC*.

Fig. 11: Effect of different B, *PR-AUC* and *ROC-AUC*.

a validation dataset.

Effect of $\mathbf{z}$*-space Dimension:* We vary the dimensionality of the latent variable $\mathbf{z}$ among 1, 2, 3, 4, 8, 16, 32, 64, 128, and 256. We only report on the effect of $\mathbf{z}$-space dimension on dataset *GD*. The results on the other datasets show similar trends. Fig. 9 shows the results w.r.t. *PR-AUC* and *ROC-AUC*.

The $\mathbf{z}$-space no longer works as a bottleneck in classical VRAE, but rather serves as a latent stochastic variable to represent the data. A large $\mathbf{z}$ enables a more powerful representation and thus better accuracy. When $\mathbf{z}$ is sufficiently large, the accuracy improvement becomes insignificant. In our proposal, the bottleneck is the robust divergences used in the objective function.

Effect of $\mathbf{h}$ *and* $\mathbf{a}$*-space Dimension:* We vary the dimensionality of the deterministic hidden states $\mathbf{h}$ and $\mathbf{a}$ among 1, 2, 3, 4, 8, 16, 32, 64, 128, and 256. For simplicity, we set $\mathbf{h} = \mathbf{a}$. We only report on the effect of $\mathbf{h}$ and $\mathbf{a}$-space dimension on dataset *GD*. The results on the other datasets show similar trends. Fig. 10 shows the results w.r.t. *PR-AUC* and *ROC-AUC*.

The results show that only the models with $\mathbf{h} = \mathbf{a} = 1$ yield bad results w.r.t *ROC-AUC* while the models with the other settings of $\mathbf{h}$ and $\mathbf{a}$ dimensionality yield the similar results. When we increase the dimensionality of $\mathbf{h}$ and $\mathbf{a}$ the model is more accuracy w.r.t *PR-AUC*.

Effect of B*:* We vary the windows size $B + 1$ among 8, 16, 32, 64, 128, and 256. We only report on the effect of $B + 1$ on dataset *GD*. The results on the other datasets show similar trends. Fig. 11 shows the results w.r.t. *PR-AUC* and *ROC-AUC*.

All methods reach the best performance when $B + 1 = 128$, indicating that $B + 1 = 128$ is the most suitable setting for our datasets.

Statistical Significance: To justify whether the accuracy improvements of the proposed methods VQRAE and BiVQRAE over the baselines are statistically significant, we conduct t-tests to test the significance of the proposed methods against

TABLE V: *GD*, p-value.

	GMMVAE	RN	CAE	RAE	BGAN	TAE	MSCRAE	RNNVAE	VAE	VRAE
VQRAE, α-divergence	4.42^{-5}	4.97^{-5}	6.26^{-5}	3.84^{-5}	4.04^{-5}	2.94^{-5}	3.89^{-5}	4.43^{-5}	8.45^{-5}	1.42^{-5}
VQRAE, β-divergence	3.38^{-5}	9.88^{-5}	5.76^{-5}	4.78^{-5}	4.85^{-5}	2.25^{-5}	4.86^{-5}	2.87^{-5}	7.47^{-5}	2.02^{-5}
VQRAE, γ-divergence	6.04^{-5}	3.13^{-5}	9.03^{-5}	4.71^{-5}	3.05^{-5}	7.77^{-5}	9.54^{-5}	8.14^{-5}	7.75^{-5}	9.40^{-5}
BiVQRAE, α-divergence	7.15^{-5}	8.97^{-5}	5.20^{-5}	9.77^{-5}	6.58^{-5}	6.38^{-5}	7.71^{-5}	5.88^{-5}	3.51^{-5}	8.30^{-5}
BiVQRAE, β-divergence	4.61^{-5}	2.42^{-5}	3.94^{-5}	6.36^{-5}	1.73^{-5}	4.10^{-5}	6.48^{-5}	6.32^{-5}	5.68^{-5}	6.16^{-5}
BiVQRAE, γ-divergence	6.49^{-5}	1.39^{-5}	1.15^{-5}	7.71^{-5}	1.53^{-5}	3.38^{-5}	5.16^{-5}	2.19^{-5}	8.80^{-5}	9.23^{-5}

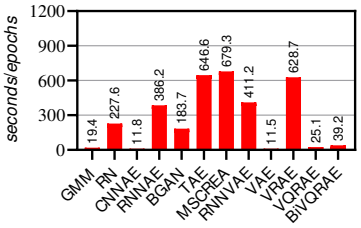
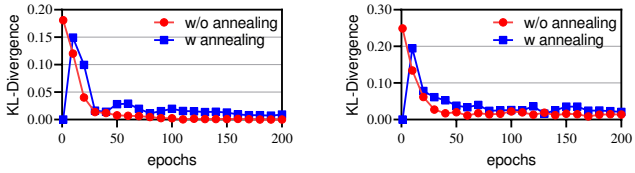


Fig. 12: Runtime performance, *seconds/epoch*, *GD*.



(a) *GD*, VQRAE. (b) *GD*, BiVQRAE.

Fig. 13: Effect of using KL-annealing trick.

baselines. We consider a null hypothesis H_0 that the mean of the anomaly scores of our methods is similar to the mean of the anomaly scores of baselines; and an alternative hypothesis H_1 that the mean of the anomaly scores of our methods is different from the mean of the anomaly scores of baselines. Based on the above, we preform a *paired, one-tail t-test* [28] to decide whether we accept or reject the null hypothesis. After performing the paired, one-tail t-test, we get a p-value [28] that represents the probability of accepting the null hypothesis H_0. If the p-value is smaller than 0.001, it shows strong evidence to reject the null hypothesis H_0, which in turns suggests that our models have statistically outperformed baselines. We only report on the study of the statistical significance on dataset *GD*. The results for the other datasets show similar characteristics. Table V shows the results w.r.t. p-value.

We observe that the p-values are all less than 0.001 for all datasets. This suggests that the accuracy improvements over baselines are statistically significant.

Runtime Performance: We compare the runtimes (*seconds/epoch*) of all methods. To achieve fair comparisons, we keep the dimensionality of (temporal) hidden states and stochastic latent variables the same across all compared methods. The window size $B+1$ is set to the default value 32. This is the default value on *GD* based on the hyperparameter selection procedure. We observe that the robust objectives are insensitive to the runtime of VQRAE and BiVQRAE. We report the mean runtime over α, β, and γ-divergence for both two methods. We only report the runtimes on dataset *GD*. The results on the other datasets show similar trends.

The results show that the runtimes of our proposed approach is competitive. Our approach achieves similar runtime as the non-recursive methods such as GMMVAE, CAE, and VAE. In comparison to recursive methods, such as RAE, MSCRAE, RNNVAE and VRAE, our models are 10-15 times faster than RAE, 17-27 times faster than MSCREA, 11-17 times faster than RNNVAE, and 16-25 times faster than VRAE. BGAN and TAE are non-recursive methods that employs CNNs and fully-connect layers in the models. However, the learning is slow because the neural network needs to optimize a minimax objective function (BGAN) or the neural network needs to compute expensive attention matrices (TAE). Our models are 4-7 times faster than BGAN and 16-25 times faster than TAE. Due to the bi-directional computation, BiVQRAE is slightly slower than VQRAE.

The testing runtimes of the two proposal models are small, i.e., less than 0.1 second, making they possible to use in online anomaly detection in streaming settings. In streaming settings, when a new observation streams in, it combines with its previous B observations to form a window of size $B+1$, which is then fed to the models to yield the anomaly score instantaneously (e.g., less than 0.1 second).

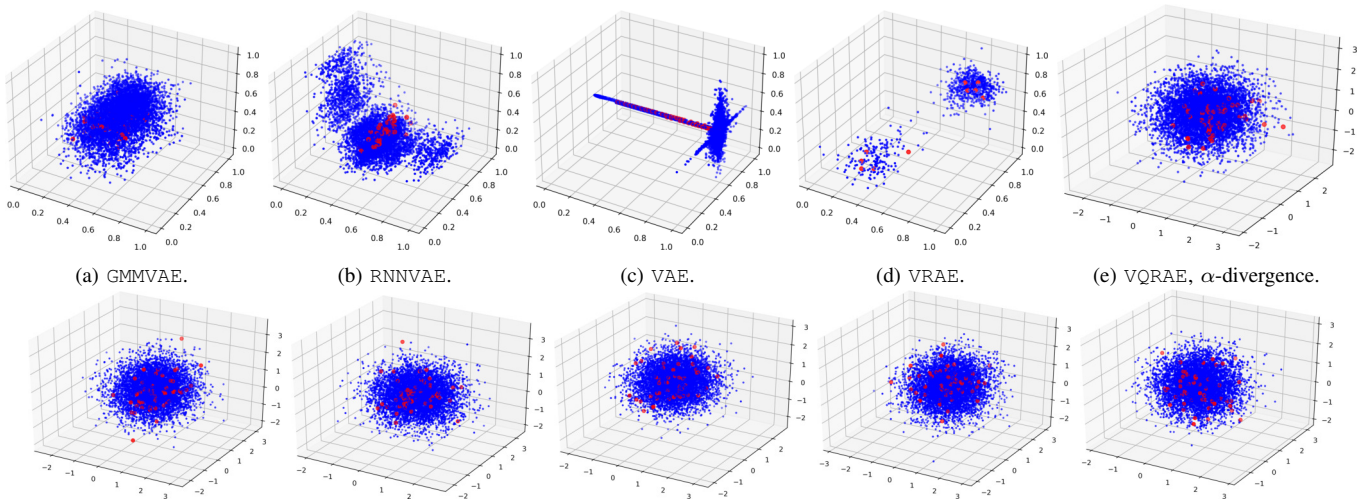
Our models can run much faster by employing pre-trained models and fine-tuning techniques instead of training every model from scratch.

Effect of KL-annealing trick: We compare the value of the KL-divergence when using versus not using the KL-annealing trick for VQRAE and BiVQRAE on dataset *GD*. The results on the other datasets show similar trends. Fig. 13 shows the result when using vs not using the KL-annealing trick for VQRAE and BiVQRAE on a an example dataset.

The results show that (i) the KL-divergence when training the model using KL-annealing trick is not equal to zero and is higher (i.e., more stochastic) than when training the model without the trick; (ii) training the model without using the KL-annealing trick is likely to yield a zero KL-divergence, which causes the model to become a deterministic function.

Visualization of $\mathbf{z}$*-space:* We visualize the latent variable $\mathbf{z}$ of RNNVAE, VAE, VRAE, VQRAE, and BiVQRAE on dataset *GD* with $\mathbf{z}$'s dimensionality is set to 3 to compare the robustness of these methods, which are reflected via the robustness of the latent spaces. The results on the other datasets show similar characteristics. Fig. 14 shows the visualization of $\mathbf{z}$-space.

For RNNVAE, VAE, and VRAE, the latent distributions were strongly distorted and perturbed by the effect of anomalies and is far from a multivariate Gaussian distribution, which is used as a prior. In other words, the latent variable captures the data



(a) GMMVAE. (b) RNNVAE. (c) VAE. (d) VRAE. (e) VQRAE, α-divergence.

(f) VQRAE, β-divergence. (g) VQRAE, γ-divergence. (h) BiVQRAE, α-divergence. (i) BiVQRAE, β-divergence. (j) BiVQRAE, γ-divergence.

Fig. 14: **z**-space visualization.

distribution of both the normal and abnormal data, which makes the reconstruction based anomaly detection less effective, thus resulting in low accuracy. The latent distribution of GMMVAE has the good shape of a mixture Gaussian distribution. However, the anomalies are strongly blended with normal data, and they tend to be close to center of the distribution, which may easily to be sampled and thus may affect the accuracy. In contrast, the latent distributions of VQRAE and BiVQRAE keep the good shape of the multivariate Gaussian distribution. In addition, the anomalies are far from the center of the distribution, which reduces their effect in sampling.

V. Related Work

Autoencoder-based Methods. CNN and RNN-based autoencoders have been used for time series anomaly detection [7], [9], [25] and other tasks [23], [45], [47]. For spatial and spatio-temporal data such as images and videos, many autoencoder-based methods exist that support anomaly detection. Zhou *et al*. [55] propose a robust autoencoder that mimics robust principal component analysis to detect anomalies in sets of images. Zhao *et al*. [53] propose a 3D CNN-based autoencoder to detect anomalies in videos. Autoencoders are also applied in denoising [43]. These methods are not able to capture latent distribution and lack generalization, thus being insufficient for modeling the unseen data.

Variational Autoencoder-based Methods. VAEs are often used to generate new data, such as text [49] and images [22]. Only a few studies attempt to employ VAEs for anomaly detection [29] and specifically for time series data [34], [42], [46] and considering robustness [2]. Xu *et al*. [46] directly employ a standard VAE for KPI time series anomaly detection. Park *et al*. [34] propose a straightforward combination of VAEs and RNNs by stacking a feed-forward VAE on top of the RNN at each time step. This combination fails to learn temporal dependencies of stochastic latent variables. VAEs with feed-forward neural networks [46] do not consider temporal dependency in time series data and lack robustness as latent variables consider both clean data and anomalies. The combination of VAEs and RNNs [34], [42] or VAEs and Transformer [14] also lack robustness and are inefficient due to recursive or attention computations. To the best of our knowledge, this paper's proposal is the first that considers three important elements in time series anomaly detection—temporal dependencies, robustness, and efficiency.

VI. Conclusion

We propose VQRAE and its bi-directional version BiVQRAE for unsupervised time series anomaly detection and address three key limitations of existing methods: poor generalization to unseen anomaly patterns, high vulnerability to anomalies, and computational inefficiency due to recurrent formulations. These models capture temporal dependencies *in parallel* by using convolution and pooling to mimic recurrent units, improving computational efficiency. From these temporal dependencies and input observations, the stochastic latent variable is learned, which makes the reconstruction more generalized. A robust objective function is integrated into the models to avoid the contamination problem of the latent space. Experimental studies offer evidence that the models are efficient and outperform strong baselines and state-of-the-art methods.

In future research, it is of interest to study advanced autoencoder structures such as autoencoder ensembles. In addition, it is of interest to consider detecting not-statistically-separate anomalies [50] to further improve accuracy and its applications in different application domains, such as transportation [18], [36].

Acknowledgments

This research was supported in part by Independent Research Fund Denmark (grants 8022-00246B, 8048-00038B), the VILLUM FONDEN (grants 34328, 40567), and the Innovation Fund Denmark center, DIREC. We also thank David Campos and Tao Wang for technical help.

REFERENCES

[1] C. C. Aggarwal, *Outlier Analysis*. Springer, 2013.

[2] H. Akrami, A. A. Joshi, J. Li, and R. M. Leahy, "Robust variational autoencoder," *CoRR*, vol. abs/1905.09961, 2019.

[3] A. Basu, I. R. Harris, N. L. Hjort, and M. Jones, "Robust and efficient estimation by minimising a density power divergence," *Biometrika*, vol. 85, no. 3, pp. 549–559, 1998.

[4] M. Berglund, T. Raiko, M. Honkala, L. Kärkkäinen, A. Vetek, and J. Karhunen, "Bidirectional recurrent neural networks as generative models," in *NIPS*, 2015, pp. 856–864.

[5] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Józefowicz, and S. Bengio, "Generating sentences from a continuous space," in *CoNLL*, 2016, pp. 10–21.

[6] J. Bradbury, S. Merity, C. Xiong, and R. Socher, "Quasi-recurrent neural networks," in *ICLR*, 2017, pp. 1–12.

[7] D. Campos, T. Kieu, C. Guo, F. Huang, K. Zheng, B. Yang, and C. S. Jensen, "Unsupervised time series outlier detection with diversity-driven convolutional ensembles," *Proc. VLDB Endow.*, vol. 15, no. 3, pp. 611–623, 2022.

[8] J. Chen, S. Sathe, C. C. Aggarwal, and D. S. Turaga, "Outlier detection with autoencoder ensembles," in *SDM*, 2017, pp. 90–98.

[9] X. Chen, L. Deng, F. Huang, C. Zhang, Z. Zhang, Y. Zhao, and K. Zheng, "DAEMON: Unsupervised anomaly detection and interpretation for multivariate time series," in *ICDE*, 2021, pp. 2225–2230.

[10] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *CoRR*, vol. abs/1412.3555, 2014.

[11] J. Chung, K. Kastner, L. Dinh, K. Goel, A. C. Courville, and Y. Bengio, "A recurrent latent variable model for sequential data," in *NIPS*, 2015, pp. 2980–2988.

[12] R. Cirstea, T. Kieu, C. Guo, B. Yang, and S. J. Pan, "EnhanceNet: Plugin neural networks for enhancing correlated time series forecasting," in *ICDE*, 2021, pp. 1739–1750.

[13] R.-G. Cirstea, B. Yang, and C. Guo, "Graph attention recurrent neural networks for correlated time series forecasting." in *MileTS19@KDD*, 2019, pp. 1–6.

[14] L. Deng, X. Chen, Y. Zhao, and K. Zheng, "HIFI: Anomaly detection for multivariate time series with high-order feature interactions," *CoRR*, vol. abs/2106.06167, 2021.

[15] J. Durbin and S. Koopman, *Time Series Analysis by State Space Methods*, ser. Oxford Statistical Science Series. OUP Oxford, 2012.

[16] M. Fraccaro, S. K. Sønderby, U. Paquet, and O. Winther, "Sequential neural models with stochastic layers," in *NIPS*, 2016, pp. 2199–2207.

[17] F. Futami, I. Sato, and M. Sugiyama, "Variational inference based on robust divergences," in *AISTATS*, 2018, pp. 813–822.

[18] C. Guo, B. Yang, J. Hu, C. S. Jensen, and L. Chen, "Context-aware, preference-based vehicle routing," *VLDB J.*, vol. 29, no. 5, pp. 1149–1170, 2020.

[19] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[20] J. Hu, C. Guo, B. Yang, and C. S. Jensen, "Stochastic weight completion for road networks using graph convolutional networks," in *ICDE*, 2019, pp. 1274–1285.

[21] J. Hu, B. Yang, C. Guo, C. S. Jensen, and H. Xiong, "Stochastic origin-destination matrix forecasting using dual-stage graph convolutional, recurrent neural networks," in *ICDE*, 2020, pp. 1417–1428.

[22] U. Jain, Z. Zhang, and A. G. Schwing, "Creativity: Generating diverse questions using variational autoencoders," in *CVPR*, 2017, pp. 5415–5424.

[23] T. Kieu, B. Yang, C. Guo, and C. S. Jensen, "Distinguishing trajectories from different drivers using incompletely labeled trajectories," in *CIKM*, 2018, pp. 863–872.

[24] T. Kieu, B. Yang, C. Guo, and C. S. Jensen, "Outlier detection for time series with recurrent autoencoder ensembles," in *IJCAI*, 2019, pp. 2725–2732.

[25] T. Kieu, B. Yang, and C. S. Jensen, "Outlier detection for multidimensional time series using deep neural networks," in *MDM*, 2018, pp. 125–134.

[26] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *CoRR*, vol. abs/1412.6980, 2014.

[27] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *ICLR*, 2014, pp. 1–14.

[28] E. L. Lehmann and J. P. Romano, *Testing statistical hypotheses*. Springer Science & Business Media, 2006.

[29] W. Liao, Y. Guo, X. Chen, and P. Li, "A unified unsupervised Gaussian mixture variational autoencoder for high dimensional outlier detection," in *BigData*, 2018, pp. 1208–1217.

[30] H. Liu, C. Jin, B. Yang, and A. Zhou, "Finding top-k optimal sequenced routes," in *ICDE*, 2018, pp. 569–580.

[31] G. Lu, W. Ouyang, D. Xu, X. Zhang, Z. Gao, and M. Sun, "Deep Kalman filtering network for video compression artifact reduction," in *ECCV*, 2018, pp. 591–608.

[32] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," *CoRR*, vol. abs/1607.00148, 2016.

[33] H. Meng, Y. Zhang, Y. Li, and H. Li, "Spacecraft anomaly detection via transformer reconstruction error," in *ICASSE*, 2020, pp. 351–362.

[34] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1544–1551, 2018.

[35] S. A. Pedersen, B. Yang, and C. S. Jensen, "Anytime stochastic routing with hybrid learning," *Proc. VLDB Endow.*, vol. 13, no. 9, pp. 1555–1567, 2020.

[36] S. A. Pedersen, B. Yang, and C. S. Jensen, "Fast stochastic routing under time-varying uncertainty," *VLDB J.*, vol. 29, no. 4, pp. 819–839, 2020.

[37] D. J. Rezende and S. Mohamed, "Variational inference with normalizing flows," in *ICML*, 2015, pp. 1530–1538.

[38] C. P. Robert and G. Casella, *Monte Carlo Statistical Methods*, ser. Springer Texts in Statistics. Springer, 2004.

[39] W. Samek, D. A. J. Blythe, K. Müller, and M. Kawanabe, "Robust spatial filtering with beta divergence," in *NIPS*, 2013, pp. 1007–1015.

[40] C. Sammut and G. I. Webb, Eds., *Encyclopedia of Machine Learning and Data Mining*. Springer, 2017.

[41] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, 1997.

[42] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *SIGKDD*, 2019, pp. 2828–2837.

[43] P. Vincent, H. Larochelle, Y. Bengio, and P. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *ICML*, 2008, pp. 1096–1103.

[44] S. Wu, W. Ren, C. Yu, G. Chen, D. Zhang, and J. Zhu, "Personal recommendation using deep recurrent neural networks in NetEase," in *ICDE*, 2016, pp. 1218–1229.

[45] X. Wu, D. Zhang, C. Guo, C. He, B. Yang, and C. S. Jensen, "Autocts: Automated correlated time series forecasting," *Proc. VLDB Endow.*, vol. 15, no. 4, pp. 971–983, 2022.

[46] H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, D. Pei, Y. Feng, J. Chen, Z. Wang, and H. Qiao, "Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications," in *WWW*, 2018, pp. 187–196.

[47] S. B. Yang, C. Guo, J. Hu, J. Tang, and B. Yang, "Unsupervised path representation learning with curriculum negative sampling," in *IJCAI*, 2021, pp. 3286–3292.

[48] S. B. Yang, C. Guo, and B. Yang, "Context-aware path ranking in road networks," *IEEE Trans. Knowl. Data Eng.*, 2020, early access.

[49] X. Yang, X. Lin, S. Suo, and M. Li, "Generating thematic chinese poetry using conditional variational autoencoders with hybrid decoders," in *IJCAI*, 2018, pp. 4539–4545.

[50] A. Zhang, S. Song, and J. Wang, "Sequential data cleaning: A statistical approach," in *SIGMOD*, 2016, pp. 909–924.

[51] C. Zhang, D. Song, Y. Chen, X. Feng, C. Lumezanu, W. Cheng, J. Ni, B. Zong, H. Chen, and N. V. Chawla, "A deep neural network for unsupervised anomaly detection and diagnosis in multivariate time series data," in *AAAI*, 2019, pp. 1409–1416.

[52] Y. Zhao, X. Chen, L. Deng, T. Kieu, C. Guo, B. Yang, K. Zheng, and C. S. Jensen, "Outlier detection for streaming task assignment in crowdsourcing," in *WWW*, 2022.

[53] Y. Zhao, B. Deng, C. Shen, Y. Liu, H. Lu, and X. Hua, "Spatio-temporal autoencoder for video anomaly detection," in *ACM MM*, 2017, pp. 1933–1941.

[54] B. Zhou, S. Liu, B. Hooi, X. Cheng, and J. Ye, "BeatGAN: Anomalous rhythm detection using adversarially generated time series," in *IJCAI*, 2019, pp. 4433–4439.

[55] C. Zhou and R. C. Paffenroth, "Anomaly detection with robust deep autoencoders," in *SIGKDD*, 2017, pp. 665–674.